Engagement Template Update Awareness

Senior Software Developer III | Architecture and implementation approach | October 2026

Design decision: keep Engagement Management authoritative, but maintain a small region-local, firm-isolated projection of template/version/decision metadata. Dashboard reads and template publication checks never synchronously rehydrate the full engagement (~1 minute). Use durable source events, Aurora PostgreSQL with row-level security, and deterministic diff-based summaries.

1. Target architecture

- Sources: shared Template Service owns version publication/eligible market target; Engagement Management owns the actual applied version and all apply/decline decisions. Attach small post-commit outbox hooks to template publication and engagement create/open/apply/decline.
- Regional derived projection: event consumers store only firmId, engagementId, templateId, market, appliedVersion, decidedTargetVersion, decisionType, sourceRevision. The record is rebuildable; no workpaper or volatile financial values are copied.
- Read path: a firm-authorized API queries the region-local index and the current eligible template offer (indexed join/read-time comparison). It returns pending status and a summary reference; no engagement hydration or synchronous inference.
- Publish/coverage path: EventBridge routes non-firm-specific template events to region-specific SQS; controlled workers evaluate per-firm impact, produce counts/heartbeats, and backfill missing metadata when necessary. Replay is safe through monotonic source revisions.
- Summary: existing JSON diff engine yields immutable deterministic facts for actual applied version -> offered version. Optional AI may rewrite these facts for readability; the displayed result, source hashes and model/prompt versions are archived.

Figure 1 (following page) shows ownership, regional event movement, tenant isolation and the read path.

Version and decision semantics

If engagement E uses v3 and the user declines v5, E still uses v3. A later eligible v7 is a new offer; the summary must be diff(v3, v7), not diff(v5, v7). Several publishes therefore coalesce into the latest eligible target without losing accumulated changes. Apply/decline is persisted by Engagement Management; this feature never applies content. Treat version IDs as identities, not sortable numbers; the template owner supplies the eligible target.

Firm-level isolation as a storage property

Structural: every engagement and decision row has a composite firm key; enable and FORCE PostgreSQL row-level security on both tables. Use a restricted DB role without BYPASSRLS, a validated SET LOCAL app.firm_id per transaction, and per-region databases. API: verify authenticated firm membership and decision permissions before DB access or forwarding an action to Engagement Management. Operations: privileged workers are scoped/audited separately. No global replication of customer engagement state.

Implementation plan and validation

Evolution of an already-running system, with no user-visible 1-minute wait

2. Phased implementation plan

Phase 1 - contracts, security, safe event capture

Agree a versioned event contract with source teams: stable event ID, firm and engagement identity, template/market, authoritative applied version, exact decision target, and monotonically increasing source revision. Write outbox atomically with each source change; publish after commit. Define regional routing, minimum privileges, service-to-service identity, decision permission checks, and RLS integration tests before enabling user traffic.

Phase 2 - projection and legacy coverage

Deploy per-region Aurora tables, indexes on (firm_id, engagement_id) and (firm_id, template_id, market), plus a shared template-offer table. Replay events into the projection. Capture metadata opportunistically when an engagement is naturally opened; enumerate existing engagement IDs and use a low-priority, explicitly capacity-limited backfill for unopened files. An old backfill snapshot must never overwrite a newer revision. Missing state is unknown/loading, not silently “up to date”.

Phase 3 - shadow evaluation and staged rollout

Before enabling indicators, compare the projection against authoritative samples per firm/product/market, including firms with zero eligible engagements. Deploy behind a feature flag by region then cohort. Keep a rollback switch for the new read path; continue collecting outbox events. Run scheduled reconciliation for missed/late deliveries. Publish progress and history metrics per firm; roll out decision links only after permission and race tests pass.

3. Testing strategy

Level / scenario	Required assertions
Domain + Java 21 JUnit	v3/v5 decline/v7 accumulation; after apply no pending; duplicate/stale events ignored; correct market and template; exact decision history once.
Security / integration	Two firms with same engagement ID; unauthorized list/decision denied; RLS blocks cross-firm SELECT/UPDATE even when API filter omitted; pooled connection context resets.
Reliability / contract	Outbox crash after source commit; at-least-once replay; consumer crash mid-batch; DLQ/redrive; late old snapshot versus new apply; withdrawn/ineligible target handled by catalog.
Performance / rollout	Large and uneven firm sizes, publish burst, index query latency, no slow loads in UI; controlled backfill and reconciliation; partial rollout and rollback.

Included code slice: immutable Java 21 contracts, a firm-scoped projection/evaluator, revision-gated in-memory adapters, seven JUnit tests, a separate offline verification runner and a PostgreSQL RLS reference schema. Infrastructure-specific adapters are deliberately omitted rather than pretending to deliver a full service. Figure 2 (following page) illustrates decline semantics and lifecycle.

Evaluation, failures and tradeoffs

Operational fitness and AI judgment

4. Evaluation and observability

- Per-publish accounting: record publish ID, applicable product/market/regions, expected firms (including firms with zero affected engagements), evaluated firms, affected-engagement counts, completion and retry/DLQ totals. Alert when an active firm has no heartbeat or no terminal outcome past the agreed evaluation window.
- Health and SLOs (proposed, to calibrate): p95 indexed-list response <300 ms; 99% of indexed update-awareness evaluations completed <60 s after eligible publish; no silent missing-firm coverage; track percentage of legacy engagements indexed separately. These are proposal targets, not numbers supplied by the assignment.
- Diagnosis: CloudWatch metrics/traces for outbox lag, SQS age, queue DLQ, projection stale/duplicate revisions, per-firm coverage, request p95, RLS authorization denials, summary evaluation results and inference spend. Correlate publishEventId -> evaluation batch -> firmId/engagementId safely without logging workpaper data.

5. Failure modes, recovery and tradeoffs

Failure / risk	Response and design tradeoff
Lost, delayed or duplicate event	Transactional outbox + at-least-once SQS; revision-gated upsert; replay and reconciliation; eventual consistency is observable.
Legacy engagement not yet indexed	Do not block list page; mark coverage unknown and queue bounded regional backfill. Never claim a false “no pending”.
Hot firm or downstream overload	Fair per-firm batches; caps on slow Engagement Management loads; queue/backoff/jitter, DLQ and alerts.
Firm data leakage or forbidden location	DB-enforced RLS + authz tests + regional storage/IAM; avoid customer content in global events and AI prompts.
LLM unavailable or unsupported claim	Use deterministic fact-based fallback; validate semantic rewrites; retain original diff, displayed text and provenance.

AI summary generation and evaluation

Use the trusted template diff service to extract structured facts, then render a rule-based readable summary. Optionally let an LLM rewrite only supplied facts, under a versioned prompt and structured-output constraint. Evaluate with a human-reviewed golden set for omissions, invented claims and terminology; fail closed to deterministic text. Archive template IDs/versions, diff hash, fact set, displayed summary, prompt/model/evaluator versions and timestamp for later audit. Do not send firm-confidential content or let AI choose eligibility or determine regulatory meaning.

Assumptions, choices, exclusions

Assume existing engagement IDs can be enumerated without hydration, events expose authoritative state after actions, and template eligibility comes from its owner. This brief specifies hundreds of engagements per firm but not global firm/product totals; do not import the previous Staff assignment’s 4k/800k/40 figures. Prefer Aurora PostgreSQL for RLS and auditable history; choose DynamoDB instead with strict partition-key IAM if query/scale warrants it. Use SQS Standard + idempotency rather than FIFO unless per-aggregate ordering becomes mandatory. Excluded: actual application, production adapters and model training.